What should you understand

Django Models — Core Concepts Reference

1 Model class

FOUNDATION

Every Django model starts the same way: a Python class that inherits from models.Model.

This inheritance is what connects your plain Python class to Django's ORM (Object-Relational Mapper). It gives your class the machinery to be saved as a database table, queried with methods like `.objects.all()`, and validated automatically.

Each model class you write maps to exactly one database table, and each instance of that class maps to one row in that table.

```
from django.db import models

class Product(models.Model):
    # fields go here
    pass
```

2 Field type

DATA MAPPING

A field type tells Django what kind of data a column holds, and how to store/validate it.

Django ships with many built-in field types, and each one is chosen to match the real meaning of the data — not just its format. Text is not always a CharField, and a date is never stored as text.

Common field types: CharField (short text), TextField (long text), DateField (calendar date), BooleanField (true/false), DecimalField (precise numbers).

```
name = models.CharField(max_length=100)
description = models.TextField()
available_from = models.DateField()
```

3 Field options

CONFIGURATION

Field options are keyword arguments that fine-tune how a field behaves, beyond its basic type.

The most common options control length, default values, and whether empty values are allowed:

• `max_length` — the maximum number of characters allowed.

• `default` — the value used when none is provided.

• `unique` — forces every row to have a different value in that column.

• `blank` and `null` — control whether the field is optional (see Validation boundary below).

```
sku = models.CharField(
    max_length=30,
    unique=True,
)
```

4 Primary key

IDENTITY

Every Django model needs a unique identifier for each row — the primary key.

If you don't define one yourself, Django automatically creates an `id` field (an auto-incrementing integer) and makes it the primary key behind the scenes.

You can always access an object's primary key through the alias `.pk`, regardless of what the actual primary key field is named. This makes your code more flexible — `product.pk` works whether the primary key is called `id` or something custom.

```
product = Product.objects.first()
product.id   # e.g. 1
product.pk   # same value: 1
```

5 Validation boundary

RULES

Not all validation happens in the same place — Django draws a clear line between the database and the form/validation layer.

• `null=True` controls whether the **database** can store NULL for that column.

• `blank=True` controls whether the field is **optional in validation** — forms and `full_clean()`.

Text fields (CharField, TextField) normally use `blank=True` alone, since Django already stores an empty string instead of NULL. Non-text fields such as DateField usually need both together to be truly optional.

```
description = models.TextField(
    blank=True, default="")

available_from = models.DateField(
    blank=True, null=True)
```

6 Readable objects

DEBUGGING

By default, printing a model instance gives an unhelpful result like <Product object (1)>.

Defining a `__str__()` method fixes this by telling Django exactly how to represent an instance as readable text — in the admin panel, in the shell, and anywhere the object is printed or logged.

This small addition makes debugging and browsing data in Django admin far easier, since you see meaningful names instead of generic object references.

```
def __str__(self):
    return self.name

# print(product) -> "iphone"
# instead of -> <Product object (1)>
```